

Lecture 12: Instruction Set Architecture (ISA), RISC & CISC

Learning Objectives

By the end of this lecture, students will be able to:

- Define Instruction Set Architecture (ISA).
 - Understand the relationship between hardware and software.
 - Explain the components of an ISA.
 - Differentiate between Machine Language and Assembly Language.
 - Identify different types of instructions.
 - Understand basic instruction formats.
 - Compare RISC and CISC architectures.
 - Recognize real-world examples of RISC and CISC processors.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn Instruction Set Architecture (ISA), understand how software communicates with hardware, compare Machine Language and Assembly Language, and explore the differences between RISC and CISC processors with practical examples."

In the previous lecture, we learned how the CPU executes every instruction using the Fetch–Decode–Execute Cycle.

But now an important question arises:

Where do these instructions come from?

How does the CPU know what "ADD", "LOAD", or "STORE" actually mean?

The answer lies in something called the **Instruction Set Architecture**, or ISA.

Think of the ISA as the language that both the CPU and the programmer agree to use. Every processor understands only the instructions defined by its own ISA.

In today's lecture, we'll learn what an ISA is, how it connects hardware and software, and why processors are commonly classified into two categories: **RISC** and **CISC**.

Let's begin.

2. What is an Instruction Set Architecture (ISA)?

Definition

Instruction Set Architecture (ISA) is the interface between software and hardware. It defines the instructions, registers, data types, addressing modes, memory model, and behavior that a processor supports.

In simple words:

ISA is the rulebook that tells the CPU which instructions it can understand and execute.

Without an ISA:

- The CPU would not know how to interpret instructions.
 - Compilers would not know how to generate machine code.
 - Programs could not communicate with the processor.
-

3. Why is ISA Important?

Suppose you write:

```
int c = a + b;
```

The compiler converts this into assembly language.

For a MIPS processor, it may generate:

```
add $t0, $t1, $t2
```

For an x86 processor, the instruction syntax is different.

Although the hardware differs, **both processors follow the rules of their own ISA.**

4. ISA as a Bridge

Software

(C++, Python, Java)

|



Compiler / Assembler

|



Instruction Set Architecture

|



CPU

The ISA acts as a bridge between software and hardware.

5. Components of an ISA

An ISA specifies several important features.

1. Instruction Set

Examples:

- ADD
- SUB
- LOAD
- STORE
- JUMP
- AND
- OR

These are the operations supported by the processor.

2. Registers

The ISA defines:

- Number of registers
- Register names
- Register sizes
- Purpose of each register

Example:

MIPS has **32 general-purpose registers**.

3. Data Types

The ISA specifies the types of data the processor can handle.

Examples:

- Integers
 - Characters
 - Floating-point numbers
 - Addresses
-

4. Addressing Modes

The ISA defines how instructions access operands.

Examples:

- Immediate
- Register
- Direct
- Indirect
- Indexed

Note: We'll study Addressing Modes in detail in a separate lecture.

5. Memory Model

The ISA defines how programs access memory.

Examples:

- Byte addressing
 - Word addressing
 - Load/Store architecture
-

6. Machine Language vs Assembly Language

Machine Language

Machine language consists of binary instructions executed directly by the CPU.

Example:

```
100011010010001000000000000000100
```

Advantages:

- Executed directly.
- No translation required.

Disadvantages:

- Difficult for humans to read and write.
-

Assembly Language

Assembly language uses symbolic instructions.

Example:

```
add $t0, $t1, $t2
```

Advantages:

- Easier to understand than binary.
- Close to machine language.

Disadvantages:

- Processor-specific.

Comparison

Machine Language	Assembly Language
Binary	Mnemonics
Difficult to read	Easier to read
Executed directly	Requires an assembler
Machine-dependent	Machine-dependent

7. Types of Instructions

An ISA usually includes several instruction categories.

Data Transfer

Moves data.

Examples:

LOAD

STORE

MOVE

Arithmetic

Performs calculations.

Examples:

ADD

SUB

MUL

DIV

Logical

Performs bitwise operations.

Examples:

AND

OR

NOT

XOR

Shift

Moves bits left or right.

Examples:

SHL

SHR

Branch and Jump

Controls program flow.

Examples:

JUMP

BEQ

BNE

8. Instruction Format

Every instruction follows a specific format.

A simplified representation:

Opcode | Operand1 | Operand2 | Destination

Example:

ADD R1, R2, R3

Where:

- Opcode = ADD
- Source 1 = R2
- Source 2 = R3
- Destination = R1

Different ISAs define different instruction formats.

9. What is RISC?

Definition

RISC (Reduced Instruction Set Computer) is a processor architecture that uses a small set of simple, efficient instructions.

Characteristics:

- Small instruction set
- Simple instruction formats
- Most instructions complete in one clock cycle
- Load/Store architecture
- Large number of registers
- Easier to pipeline

Examples:

- MIPS
 - ARM
 - RISC-V
-

10. What is CISC?

Definition

CISC (Complex Instruction Set Computer) is a processor architecture that provides a large set of more complex instructions.

Characteristics:

- Large instruction set
- Variable-length instructions
- Some instructions perform multiple operations
- More complex decoding

Examples:

- x86
 - x86-64
-

11. RISC vs CISC

Feature	RISC	CISC
Instruction Set	Small	Large

Instruction Length	Usually fixed	Often variable
Execution	Many simple instructions	Fewer, more complex instructions
Hardware Complexity	Simpler	More complex
Registers	Usually more	Usually fewer (historically)
Pipeline Friendly	Yes	More challenging

12. Real-World Examples

RISC

- ARM processors in smartphones and tablets.
 - Apple Silicon (implements the ARM ISA).
 - MIPS (commonly used in education).
 - RISC-V (open standard gaining popularity).
-

CISC

- Intel Core processors.
- AMD Ryzen processors.

Most desktop and laptop computers use x86/x86-64 processors.

13. Why Are We Learning MIPS?

Many universities teach **MIPS** because:

- It has a simple RISC design.
 - Instructions are easy to understand.
 - It helps students learn processor organization without unnecessary complexity.
 - It clearly demonstrates registers, memory, and instruction execution.
-

14. Example: Same Task

High-level code:

```
int z = x + y;
```

MIPS:

```
add $t0, $t1, $t2
```

Conceptually:

Register1 + Register2

↓

ALU

↓

Register3

The exact syntax differs on other ISAs, but the idea remains the same.

15. Real-Life Analogy

Imagine two people traveling.

RISC

One small, simple step at a time.

Walk

↓

Turn

↓

Open Door

↓

Enter Room

CISC

One complex instruction.

"Go into the room."

Internally, that complex action still consists of many smaller steps.



Lecture Summary

Today, we studied the Instruction Set Architecture (ISA), which defines the instructions and behavior a processor understands.

We explored the relationship between hardware and software, compared Machine Language and Assembly Language, and learned about the main components of an ISA.

Finally, we compared the two major processor architectures—RISC and CISC—and discussed why MIPS is widely used for learning Computer Organization and Assembly Language.

In the next lecture, we'll begin the Assembly Language section by learning **Number Systems**, including Binary, Decimal, Octal, and Hexadecimal. We'll also cover number conversions and explain why binary is fundamental to digital computers.

Thank you for watching, and I'll see you in the next lecture.
